

Quantum Benders Decomposition for Train Rescheduling Problems

Moritz Heidtmann, Andrei Zubarev, Daniel Kratzer, Kilian Krüger, Emanuele Poggi
LMU Munich

Abstract— Railway operators must restore safe, minimally delayed timetables within minutes after disruptions, yet traditional monolithic mixed-integer models scale poorly as hundreds of trains interact on dense networks. We address this by proposing a quantum-hybrid Benders decomposition that decouples continuous timing from combinatorial precedence decisions. Our framework translates industrial track and schedule data into fixed and selectable lower-bound constraints, then uses a Z3-based satisfiability subproblem with “bucketed” deviation predicates as assumptions. Minimal unsatisfiable cores serve as Benders cuts in a set-cover master problem which we solve via a Quantum Alternating Operator Ansatz implemented with intersection-aware, multi-controlled bit-flip mixers. We execute the circuit on IBM’s superconducting backends, up to 133 qubits, and evaluate on Siemens’s real-world instances, with six trains over six stations. We benchmark against both classical Benders decomposition approaches and a fully centralized classical solver. We show that our method converges in significantly fewer iterations than classical baselines, while still producing feasible timetables that preserve all safety margins and operating within current quantum available resources. The primary limitation remains the extended runtime on quantum hardware. Our findings highlight the need for further research into noise-resilient ansatz design and backend-specific compilation techniques to fully unlock the potential of quantum-hybrid decomposition methods for rail rescheduling in the NISQ era.

I. INTRODUCTION

In 2024 alone, the German railway network operated by Deutsche Bahn had a train-path demand of 1.103 billion kilometers and 160.1 million station stops [1, p.139]. The scheduling of trains, including their routes and stops is therefore a highly complex problem. Multiple factors further constrict the scheduling of trains. These constraints need to be met, to ensure a running train network and further add to the aforementioned complexity. Even small delays of trains can induce a domino effect that cascades through a railway network resulting in further disruption. Having a well-run network is critical for passenger satisfaction and economic activity.

The Train Rescheduling Problem (TRP) is NP-hard [2]. This complexity results from the high number of potential solutions for trains competing for the same track segments. Deciding the optimal order of multiple trains is creating a solution space that grows exponentially with the number of conflicts.

Approaches, like Benders decomposition [3] have been promising, compared to other classical alternatives. Lowering the runtime of the algorithms to compute a solution in a timely manner is crucial to react quickly to arising conflicts. In our work, we intend to build on Benders decomposition

and achieve further improvements with the help of quantum algorithms.

Quantum computing has been an emerging field with scientific advancements enabling the mapping of some real world scenarios to quantum algorithms and solving them subsequently. Some quantum algorithms like the Quantum Approximate Optimization Algorithm (QAOA) have been applied to train rescheduling. The underlying hypothesis is that these quantum algorithms can produce high-quality solutions for problems that are challenging for classical algorithms.[4] However most of the time, the approach was monolithic and therefore limits scalability.

This paper approaches the TRP by leveraging Benders Decomposition to partitioning it into a master problem and a subproblem. This decomposition isolates the complex part within the master problem taking the form of a Set Covering Problem (SCP), a NP-hard problem well suited for quantum computing algorithms. We address SCP using the Quantum Alternating Operator Ansatz (QAOA+) which will be detailed later in this paper and is a powerful variant of QAOA.

II. BACKGROUND

A. Microscopic Train Rescheduling Problem

Consider a railway network N . Let N be given as a graph with edges representing segments that only one train can access at once. Some of those segments belong to station platforms. An initial schedule indicates when a train should be at which station in N . Now, suppose there is a conflict in the given data, i. e. the schedule cannot be fulfilled. That could be the case if the schedule is based on an older network but now some segments are blocked due to renovation work or storm damage. A TRP is an optimization problem that tries to create a new conflict-free schedule. Its goal is to minimize the overall deviation from the initial one. To model the problem, we can directly follow the model that Leutwiler et al. proposed [5], [3]. They consider events ‘train a entering the segment p ’ as tuples $(a, p) \in E$ with E being the set of all events. For each $i \in E$, they introduce time variables t_i for the start time of the event i . The problem can then be transferred to a set of constraints.

One part of these constraints models the routing and ordering of trains, speed limits, and routing decisions if there is more than one path for a train to take. As it is not the focus of this work, we do not go into detail on constructing those constraints. The other part limits the amount of deviation from the original schedule. For that, they further introduce

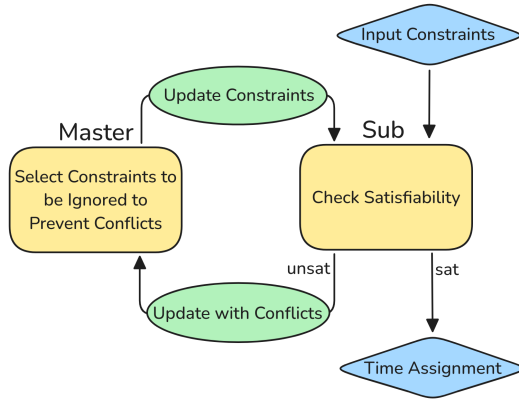


Fig. 1. Benders Decomposition Workflow

new variables $\delta_i \leq \delta_{max}$ for each event of interest with δ_{max} being a given maximal deviation. For this work, let events of interest be only events on station platforms. With those additional variables and the latest time f_i for the event i we get constraints of the form:

$$\delta_i - t_i \geq -f_i$$

A solution of the TRP is found by minimizing the sum over all deviation variables while satisfying all the other constraints.

B. Benders Decomposition of the TRP

By discretizing the deviation variables one can perform a Benders decomposition of the problem, also done by Leutwiler et al. [3]. To do this, we modify the constraint set so that each deviation constraint is replaced with the series of constraints

$$(\delta_{i,k} - t_i \geq -f_i)_{0 \leq k \leq K}$$

with $K \in \mathbb{N}$ and constants $\delta_{i,k} = k \frac{\delta_{max}}{K}$.

To solve the TRP we now consider a masterproblem and a subproblem. In the subproblem, we just check if the modified constraint set is satisfiable. If it is, then it returns a model of the given problem as a solution to the problem. If there is a conflict, it computes a cut for the master based on an infeasible subset of the constraints. For that subset, we only consider deviation constraints. The master then selects which constraints to ignore in the next iteration to prevent all conflicts that the subproblem has encountered so far. Thus, the master is an instance of a SCP with the conflicts acting as elements and the conflicting constraints as subsets containing them. After that we try to solve the subproblem again, but ignoring constraints that correspond to subsets selected by the master. That procedure is performed several iterations until the subproblem satisfies. In this work, we refer to these iterations as Benders iterations. The general decomposition and iteration workflow can be seen in Figure 1.

C. QAOA+

The QAOA+ (Hadfiel et al. [6]) is a generalization of the QAOA (Farhi et al. [4]). The main difference is that

QAOA+ only explores the subspace of the Hilbert space that only includes states representing a feasible solution for the problem to solve. For example, when solving set covering, the solution subspace are only states which represent a Set Cover. That allows us to avoid implementing penalty terms in the cost Hamiltonian. As input state a trivially feasible state has to be prepared instead of a superposition of all states. To remain in the feasible subspace, the mixer Hamiltonian H_m must implement a transfer from one feasible state to another. Furthermore, for two given feasible states, there have to exist parameters $(\alpha_i)_{0 \leq i \leq n-1}$ to transfer from one state to the other in n steps by iteratively applying $H_m(\alpha_i)$.

The workflows of QAOA+ and QAOA are the same. A classical algorithm is used to optimize the parameters of the constructed quantum circuit, aiming to minimize a certain cost function. With optimized parameters, the circuit is called once more to compute a final solution.

III. RELATED WORK

The problem of train rescheduling can be naturally reformulated as a SCP, for which both classical and quantum solution techniques have been explored. This section reviews the most relevant lines of research and positions our contribution accordingly.

A. Set Covering and Classical Approaches

SCP is a foundational NP-hard problem with wide applicability, including crew scheduling, facility location, and timetabling. Classical exact methods such as Beasley's three-stage algorithm [7] and the approach by Balas and Ho [8] use a combination of dual ascent, cutting planes, and problem reduction. These methods provide tight bounds but scale poorly with instance size.

On the heuristic side, Chvátal's greedy algorithm [9] remains a theoretical and practical baseline, offering a logarithmic approximation guarantee. However, these classical approaches either scale super-polynomially or sacrifice optimality—making them less suited for large, real-time scheduling problems.

B. Benders Decomposition

Benders decomposition [10] is a powerful technique to tackle large-scale mixed-integer programs by separating complicating variables. Leutwiler et al. [5], [3] applied this framework to microscopic railway timetabling, effectively reducing complexity by handling conflicts via subproblem feasibility. In our setting, this decomposition allows each subproblem to be mapped to a compact SCP instance, which is particularly suitable for quantum treatment.

C. Quantum Algorithms for Combinatorial Problems

Variational Quantum Algorithms (VQAs) have emerged as a leading class of heuristics for solving combinatorial optimization problems on current quantum hardware. These hybrid algorithms leverage classical optimizers to train parameters required for a quantum circuit, or ansatz with the goal of

preparing a quantum state that encodes the solution to the problem. Grange et al. [11] have provided a comprehensive introduction to VGA, including discussions of the roles of the quantum ansatz, classical optimizers, and guiding functions that connect them. In particular, QAOA has been covered.

Farhi et al. [4] introduced QAOA as a hybrid classical-quantum variational framework for solving discrete optimization problems. For structured problems like MaxCut or SCP, QAOA achieves competitive approximations even at low depth. Its structure, alternating between cost and mixing unitaries, fits naturally into the Benders subproblem formulation.

D. Constraint-Preserving Quantum Methods

Hadfield et al. [6] extended QAOA by introducing the QAOA+, which restricts the evolution to a feasible subspace using problem-specific mixers. This avoids the need for penalty terms in the cost Hamiltonian and results in shallower circuits. Our work builds directly on this idea, designing mixers that guarantee feasibility for SCP instances derived from Benders decomposition.

E. Applications in Rail Scheduling

Grange et al. [11] demonstrated that railway scheduling problems can be reformulated as SCPs and solved with QAOA and Grover search. However, their approach treats the scheduling problem monolithically, limiting scalability. In contrast, the method we use, decomposes the full schedule into smaller SCP subproblems.

Another practical example on how QAOA can be applied to solve railway timetabling was also provided by Grange [12] in a separate paper. The work maps the large scale industrial problem, which was originally encoded as an integer linear program, to SCP and Extended Bin Packing. A relevant part of this work is comprised of the investigation into the performance of different penalty approaches such as PUBO and QUBO. Furthermore it covered the impact of the choice of QAOA parameters on the final quality of solutions. The findings show, that QAOA can find optimal solutions for smaller sized problems and that its performance is substantially to the problem formulation. Based on that, the performance of a QAOA approach is highly dependent on how constraints are being formulated. This paper highlights practical hurdles in applying QAOA directly and underscores the importance of reformulating real world problems to enable them being solved by QAOA. This provides a valuable foundation for our approach which aims to structure the problem differently.

Domino et al. [13], who tackled a stochastic problem on the Baltimore Light RailLink, provided a different approach to train rescheduling. Their work focused on using the inherent noise of Noisy Intermediate-Scale Quantum technology (NISQ) to model random disturbances in train transit timelines. The problem was formulated as a QUBO. It was solved using quantum annealing on a D-Wave machine and a QAOA implementation on an IonQ computer. This serves as a comparison between two leading quantum computing approaches for the same railway problem. Their work found, that hardware

specific characteristics of quantum computers have a critical influence on performance. This direction of research used device noise for stochastic modeling, is a difference to our approach where we use Benders decomposition to manage complexity and ensure schedule feasibility.

F. Summary

In summary, our work combines Benders decomposition with QAOA+, leveraging domain structure and feasibility-preserving quantum mixers to address the scalability limitations of existing quantum and classical approaches. To our knowledge, this is the first application of QAOA+ to a Benders-decomposed industrial scheduling problem.

IV. METHODOLOGY

As described in Section II, we perform a Benders decomposition on the TRP. In the following, we give a detailed description on how we solve the subproblem and the SCP of the master problem. Instead of using classical algorithms for the SCP part, we present an approach that utilizes quantum computing to achieve a potential speed-up compared to classical approaches.

A. Satisfiability Solver

Given a tentative master solution $\mathbf{x} \in \{0, 1\}^{|\mathcal{I}_S|}$, we must decide whether the associated constraint set $\mathcal{S}(\mathbf{x})$ is feasible. We phrase this check as a *pure* (no objective) satisfiability problem in the reals and encode it in the Z3 SMT solver [14]. The resulting model plays the role of the *sub-problem* in Benders decomposition.

Variables:

- Arrival time $t_{i,s} \in \mathbb{R}_{\geq 0}$ of train i at network vertex s ; the unique origin event is set to $t_{origin} = 0$.
- Auxiliary *deviation* variable $\delta_{i,s} = t_{i,s} - f_{i,s}$ used for the global budget.
- Predicate family $\{p_{i,s,k}\}$ that guards the k -th deviation bucket of event (i, s) .

Constraint encoding: Let $\underline{f}_{(i,s),(i,s')}$ and $\underline{f}_{(i,s),(j,s)}$ denote the fixed intra-train and selectable inter-train lower bounds produced by the translator. We encode them as

$$t_{i,s'} - t_{i,s} \geq \underline{f}_{(i,s),(i,s')} \quad \forall ((i,s),(i,s')) \in \mathcal{F},$$

$$\bigvee_{c \in C_d} \bigwedge_{((i,s),(j,s)) \in c} t_{j,s} - t_{i,s} \geq \underline{f}_{(i,s),(j,s)} \quad \forall d \in \mathcal{D}.$$

Here $\mathcal{F}$ is the set of fixed precedences and each decision d provides a choice set C_d of mutually exclusive routes.

For stations with multiple candidate platforms we add one multi-input XOR constraint to ensure exactly one corresponding $t_{i,s}$ is non-zero.

Deviation buckets are handled such that for each discretisation level k we guard the upper bound

$$p_{i,s,k} \implies |t_{i,s} - f_{i,s}| \leq k \frac{\delta_{\max}}{K}, \quad k = 0, \dots, K-1,$$

and the always-active clause $t_{i,s} \leq f_{i,s} + \delta_{\max}$ caps the loosest bucket ($k = K$).

Assumption-based solving: All predicates $p_{i,s,k}$ are declared as Z3 Bool atoms; the sub-problem is invoked through `solver.check(A)` with a *dynamic* assumption list $A(B)$:

- initial call: A contains *only* the tightest bucket $p_{i,s,0}$ for every event;
- later calls: A contains all buckets not disabled by the master.

If Z3 returns `unsat`, its `unsat_core()` is a subset of A and therefore references only deviation predicates. These are translated one-to-one into set-cover *elements* for the master.

Algorithm 1: Z3 solver

```

Input: disable-set  $B \subseteq \{p_{i,s,k}\}$  (may be  $\emptyset$ )
Output: SMT flag, model  $t$ , unsat core  $C$ 
1  $A \leftarrow \emptyset$ ;
2 foreach event  $(i, s)$  do
3   for  $k = K-1$  downto 0 do
4     if  $p_{i,s,k} \in B$  then break;
5      $A \leftarrow A \cup \{p_{i,s,k}\}$ ;
6   end
7 end
8  $res \leftarrow \text{solver.check}(A)$ ;
9 if  $res = \text{sat}$  then return (true, model  $\rightarrow$  dict,  $\emptyset$ );
10 else if  $res = \text{unsat}$  then  $C \leftarrow A \cap \text{solver.unsat\_core}()$ ;
11 if  $C = \emptyset$  then raise EMPTYCOREEXCEPTION;
12 return (false,  $\emptyset$ ,  $C$ );
13 else
14   raise run-time error ("unknown");
15 end

```

Complexity: Each precedence edge contributes one linear constraint and at most one literal. Overall we add

$$|\mathcal{F}| + \sum_{d \in \mathcal{D}} \max_{c \in C_d} |c| + (K+1)|\mathcal{I}_\delta| + |\mathcal{X}|$$

constraints (where $\mathcal{X}$ counts XOR ambiguity constraints) and $K|\mathcal{I}_\delta|$ Boolean variables. Z3 handles the Boolean-linear mixture with an incremental CDCL(T) solver.

B. QAOA+ Solver for SCP

The SCP is a classical combinatorial optimization problem where the objective is to select a subset of sets that together cover all required elements, with minimal cardinality or cost. In our formulation, we represent each available subset (i.e., column of the instance matrix) by a single qubit, and use the QAOA+ to sample promising solutions. This binary representation aligns well with quantum hardware: each bit in the output string corresponds to selecting (1) or ignoring (0) a specific subset.

1) *Following Hadfield's QAOA+:* Our quantum formulation for the SCP is informed by the general QAOA+ framework proposed by Hadfield et al. [6]. In particular, we follow their encoding of feasible configurations as bitstrings where each bit represents the inclusion of a subset, and adapt their mixer design for constrained optimization. While the original QAOA+ framework focuses on feasibility-preserving mixers, we begin with a simplified cost-only Hamiltonian to analyze baseline behaviour before enforcing hard constraints via auxiliary constructions.

2) *Qubit Encoding and Problem Mapping:* Given an SCP instance as a binary matrix with rows representing elements and columns representing subsets, we allocate one qubit per column. Thus, a problem with n subsets results in an n -qubit quantum circuit. Each possible bitstring sampled from the final quantum state corresponds to a candidate subset cover.

3) *Cost Hamiltonian:* Our cost Hamiltonian is designed to encourage the selection of fewer subsets. We use a simple linear formulation based on the PauliZ operators:

$$H_C = \sum_{i=0}^{n-1} Z_i$$

This structure penalizes active qubits (i.e., selected subsets), as $Z|1\rangle = -|1\rangle$. In our implementation, we minimize the number of active qubits by directly optimizing the output bitstrings with the fewest ones, instead of minimizing the expectation value of a cost Hamiltonian. Notably, we do not encode the coverage constraint in the cost Hamiltonian — instead, it is enforced implicitly via the mixer design. This version of the cost Hamiltonian focuses purely on minimization of the cover size and serves as a baseline for observing QAOA's performance in unconstrained settings.

4) *Mixer:* As mentioned in Section II the mixer $H_m(\alpha)$ ensures that exploration is only performed in a feasible subspace such that, when applied to a feasible state $|\phi\rangle$, the state $H_m(\alpha)|\phi\rangle$ is still feasible, that is, in the case of Set Covering, it represents a set cover. For the implementation of the mixer Hamiltonian we stick closely to the description of a controlled Bit-Flip-Mixer suggested by Hadfield et al. (see Section A.2.5 [6]) to use for Set Covering.

$H_m(\alpha)$ is divided into partial mixers $H_{m,S_j}(\alpha)$, one for each subset S_j . Those partial mixers are built as follows: To obtain feasible states, the bit x_j corresponding to S_j should only be flipped if all its elements are already covered by other subsets according to the input state of the mixer. That is the case if for each element in S_j another subset is currently selected that contains this element. To implement this with quantum gates, we need an ancillary qubit q_i for each element $i \in S_j$. On each of these ancillas, we encode the following logical formula:

$$\bigvee_{l: i \in S_l \wedge l \neq j} x_l$$

To encode a logical *Or*, we can use a scaled version of the circuit in Figure 2.

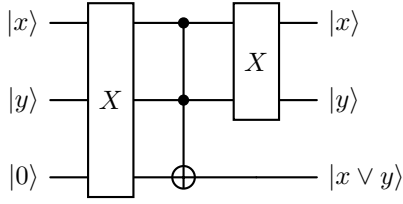


Fig. 2. A small circuit implementing a logical Or with quantum gates.

After that, we can apply a controlled $RX(\alpha)$ -Gate on the bit representing S_j with all the ancillas corresponding to the elements in S_j used as controls. That gate performs the actual mixing operation and is therefore parameterized. To clean up the ancillas in preparation for the next partial mixer, we again apply the *Or*-circuits we used before. The constructed partial mixers are then applied sequentially.

Hadfield et al. stated that the order in which we apply the partial mixers could affect the optimization speed. For now, we apply them in the same order that they were added to the SCP-Instance during Benders iterations for simplicity. To optimize circuit depth, we can leave out the majority of X -gates of the *Or*-circuit by applying X -gates only at the beginning of $H_m(\alpha)$, at the end, and around the $RZ(\alpha)$ -Gates. The *Or*-circuit is then only implemented with a single multi-controlled X -Gate. With this depth optimization, we end up with a mixer circuit such as in Figure 3.

5) *Circuit Structure*: Given that only a small fraction of states in the Hilbert space correspond to valid set covers, we adopt the QAOA+ framework [6], which uses feasibility-preserving mixer Hamiltonians to restrict exploration to the feasible subspace. This allows the circuit to sample only valid solutions throughout optimization. Our cost Hamiltonian remains simple and unconstrained, focusing solely on minimizing the number of selected subsets. Constraint enforcement is shifted entirely into the mixer design.

This is especially useful for Set Covering, where feasible solutions form a tiny fraction of the Hilbert space. As Hadfield et al. point out, feasibility-preserving mixers “confine the evolution of the quantum state to the feasible subspace,”[6] helping avoid invalid samples and making shallow circuits more effective — a good fit for NISQ-era limitations.

Each QAOA+ layer consists of a cost unitary followed by a mixer unitary, repeated for a fixed depth p . The final circuit has the structure:

$$(e^{-i\beta_p H_M} e^{-i\gamma_p H_C}) \dots (e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C})$$

Following Hadfield’s QAOA+ design, we begin from a trivially feasible initial state, such as the all-ones bitstring, which guarantees full coverage. Since our mixer only allows transitions within the feasible subspace, this choice ensures that all sampled states during the optimization remain valid set covers.

In our implementation, the cost Hamiltonian is realized using single-qubit $RZ(2\gamma)$ rotations applied to each qubit.

This corresponds to the operator $H_C = \sum_i Z_i$, which penalizes active qubits (i.e., selected subsets). The mixer unitary, responsible for maintaining feasibility, is described separately above.

6) *Parameter Optimization*: The parameter optimization can be seen as the classical part of QAOA+. It serves as the integral part to optimize the parameters γ and β which are used in the cost and mixer hamiltonian respectively. The goal of the optimization is to find the optimal parameter set that minimizes the expected size of the set cover.

Scipy [15] is used to apply the The Constrained Optimization By Linear Approximation (COBYLA) method, a gradient free optimization algorithm.

For each set of parameters proposed by COBYLA, the cost function is called with these parameters. Then, the quantum circuit is executed for a fixed number of shots, which returns a counts value, representing the outcomes of that circuit with the proposed parameters for each run. For each measured bitstring, the number of ‘1’s is counted, which represents the size of the potential set cover, and multiplied by the number of times that string was measured. The sum of these values is the total cost of that run. A lower value means, that the parameters proposed resulted in smaller set covers on average. This total cost is returned to the COBYLA optimizer which uses the value to further optimize the parameters. This process is continued until a stopping criterion is met. The final output is the set of parameters $\{\gamma, \beta\}$ that resulted in the lowest cost found by the optimizer.

7) *Scalability Estimations*: Let’s take a look at how the algorithm scales in terms of qubit count and circuit depth and what that means in the context of SCPs in our Benders decomposition of TRP.

The number of qubits is determined by the mixer. In general, we need one qubit for each subset and one ancillary qubit for each element. In our use case, the size of the SCP to solve increases with each Benders iteration. Until the constraint set is satisfied, a new conflict is found each iteration; hence, a new element is added to the SCP. That results in at least one additional qubit per iteration. Predicting how many additional subsets we get per iteration is more difficult: conflicts can arise between constraints of an unknown number and between constraints that are already included in the SCP as subsets. That is why we are not able to predict exact increases for the qubit count that are caused by additional subsets. But we will have a detailed look on how qubit counts behave during our hardware experiments in Section VI.

To quantify the cost of quantum algorithms, Nielsen et al. [16] propose circuit depth as a potential measure. As a layer-based algorithm, the overall depth is dependent on the number of layers, of course. Hadfield et al. [6] already provide the following estimates on layer depth: initial state preparation at the beginning and the cost Hamiltonian are of constant depth 1. For each partial mixer H_{m,S_j} , they state that depth depends linearly on D_j being the number of all subsets that intersect with the subset S_j . Moreover, since we will need as many partial mixers as there are subsets in the SCP, the depth

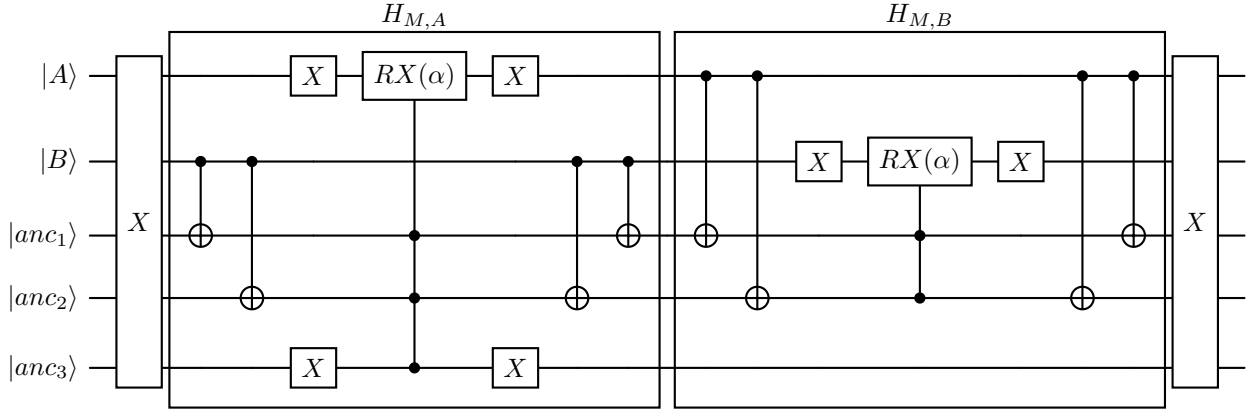


Fig. 3. A mixer implementing the set covering instance $\{A = \{1, 2, 3\}, B = \{1, 2\}\}$

of the whole mixer additionally depends on the number of all subsets. The structure of SCP instances in the context of Benders decomposition was not yet examined enough to give more detailed predictions about circuit depth of the mixer for now.

V. EXPERIMENTAL SETUP

The goal is to evaluate whether a feasibility-preserving quantum ansatz can effectively solve structured Set-Covering subproblems arising in industrial train rescheduling.

Our workflow starts with raw industrial train scheduling data and ends with a feasible solution to a constrained optimization problem. A custom parsing module extracts possible rerouting options and temporal precedence constraints. These are used to identify space-time conflicts between trains, which are then encoded into a binary matrix: each row represents a conflict, and each column represents a possible routing decision. This matrix defines the SCP instance that our solvers operate on.

Before solving, we check feasibility using a Z3 SMT solver. Only conflicts that survive this check are passed into the SCP. Internally, SCPs are wrapped in a flexible object format that can be passed to both classical and quantum solvers without modification.

All experiments are reproducible and modular: solvers can be swapped, data files can be replaced, and parameters can be adjusted easily. Visualization of output distributions and selected bitstrings allows comparison between approaches. While this setup is still exploratory, it provides a flexible testbed for evaluating hybrid strategies in the context of combinatorial optimization with industrial relevance.

In the following we detail: (A) the dataset used, (B) the four solver strategies (classical and quantum), and (C) the performance metrics we record.

A. Data

We benchmark on 19 TRP instances derived from a Norwegian regional network dataset provided by Siemens. Each instance corresponds to a disruption scenario—e.g. a temporary network shutdown or service interruption—characterized

by an earliest access time, after which all trains may re-enter the network.

- **Network topology:** 40 connector nodes and 47 track segments, each annotated with maximum speed and segment length.
- **Train-Station schedules:** 6 stations and 6 trains per instance, each defined by ordered stop events (arrival/departure times) at specified stations.

These inputs are parsed into Z3 precedence-bounds and an element-by-subset conflict matrix for the SCP subproblem. We map each scenario’s earliest access time (from 15:32:00 to 15:49:00) to a difficulty level 0–18; later access times increase difficulty because trains have less time to reach their first stops, yielding more potential conflicts.

B. Solvers evaluated

To evaluate the impact of the quantum-enhanced master-iteration against both classical and fully centralized baselines we include four solver strategies.

All methods operate on identical SCP inputs and risk-platform settings to ensure a fair and reproducible comparison.

- **Centralized Solver.** Solves the full TRP in one Gurobi 9.1 model [17], including continuous timing variables, bounded deviation variables, binary selectors for ambiguous stops, and all precedence and unambiguity constraints. It does not use Benders decomposition and serves as a gold-standard for total deviation at the cost of larger model size and longer runtimes.
- **Greedy Heuristic (Bender Master).** Within the Benders loop, this baseline uses SetCoverPy [18] to iteratively select the deviation predicate covering the most unmet conflicts. It converges in few iterations but may yield suboptimal delay.
- **Gurobi MIP (Bender Master).** Also inside the Benders framework, this uses Gurobi 9.1 to exactly solve the master’s Set-Covering subproblem as a binary LP (one variable per deviation predicate, one constraint per conflict), with default MIPGap=0 and no time limit.

- **QAOA+ (Bender Master).** A quantum-hybrid master solver implemented in PennyLane:
 - Feasibility-preserving mixer ansatz with depth $p = 2$.
 - $2p$ parameters $\{\gamma_i, \beta_i\}$ optimized via COBYLA (maxiter=10) on shot-based cost estimates (shots=10).
 - Circuits transpiled at optimization_level=1, executed on both on *AerSimulator* and IBMQ’s *ibm_torino* (128 shots, up to 5 retries).

C. Reported metrics

We assess each solver’s performance using the following key metrics:

- **Wall-clock runtime.** The total end-to-end time (in seconds) to complete the Benders decomposition (or centralized solve), including both the master-loop solves and any quantum circuit execution. Any queue delays on quantum hardware are subtracted so that reported times reflect pure computation. This metric captures practical runtime overhead and is essential for assessing the viability of each approach in a real-time scheduling setting.
- **Number of Iterations.** The number of master-loop cycles performed before the subproblem reports feasibility or a predefined iteration limit is reached. Fewer iterations indicate faster convergence of the decomposition and more effective relaxation proposals.
- **Total Deviation.** Sum of all train delay values (in seconds) in the rescheduled timetable. Measures how much the rescheduled plan deviates from the original schedule.
- **Maximum Qubit Usage** – The peak number of qubits (problem + ancilla) required in any quantum iteration. This quantifies the hardware size needed to run the feasibility-preserving mixer ansatz in practice.

We implemented the hybrid Benders–QAOA+ framework, classical baselines, data translation, and experimental workflows in Python and make our code available on GitLab¹

VI. EVALUATION

We compare the four solver strategies on the metrics defined above. Whenever repeated runs were performed we report the mean values with one-standard-deviation error.

A. Runtime

Classical runtimes grow predictably with instance difficulty (Figure 4). The greedy heuristic incurs minimal overhead, making it the fastest across all levels. Exact optimization on the master problem (Gurobi) adds moderate cost, while solving the full Train Rescheduling Problem in one centralized model leads to the highest runtimes for medium to hard instances.

Classical runtimes grow predictably with instance difficulty (Figure 4). As also noted by Leutwiler et al. [3], the greedy heuristic incurs minimal overhead, making it the fastest across

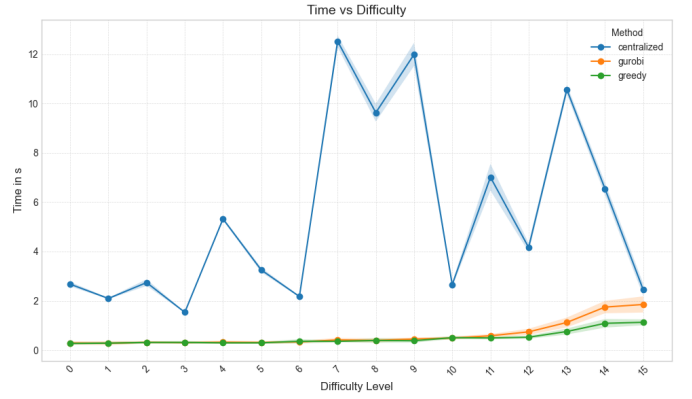


Fig. 4. Classical solver runtime vs. problem difficulty.

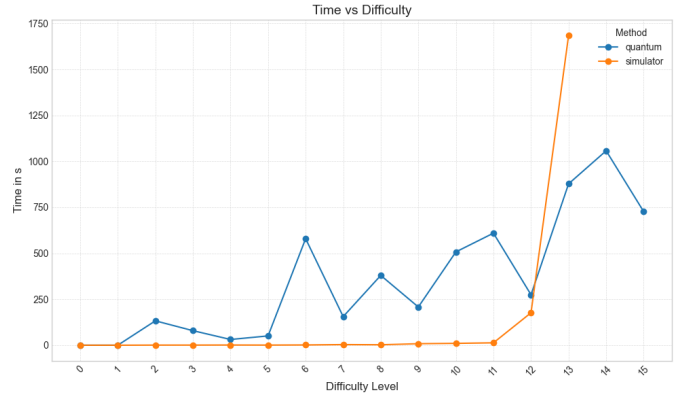


Fig. 5. Quantum solver runtime vs. problem difficulty on simulator (orange) and hardware (blue).

all levels. Exact optimization on the master problem (decomposed via Gurobi) adds moderate cost, while the centralized solver exhibits the highest runtimes.

Quantum solver runtimes are substantially higher (Figure 5). On the local simulator, execution time increases sharply as circuit depth, shot count, and optimizer iterations grow with difficulty. Remote hardware runs incur additional cost from transpilation, network latency, and job execution overhead, revealing the current NISQ bottlenecks in practical quantum execution.

B. Deviations

Solution quality across methods is summarized in Figure 6. The centralized solver provides the benchmark minimum deviation. Both decomposed solvers show larger delays on hard instances. Notably QAOA+ deviations nearly coincide with the latter for low to medium difficulties, with only marginal excess delay at the highest levels.

C. Iteration Count

Convergence behavior differs markedly between classical and quantum-hybrid approaches. As illustrated in Figure 7, QAOA+ (both simulator and hardware) converges in far fewer

¹<https://tinyurl.com/QuantumBender>

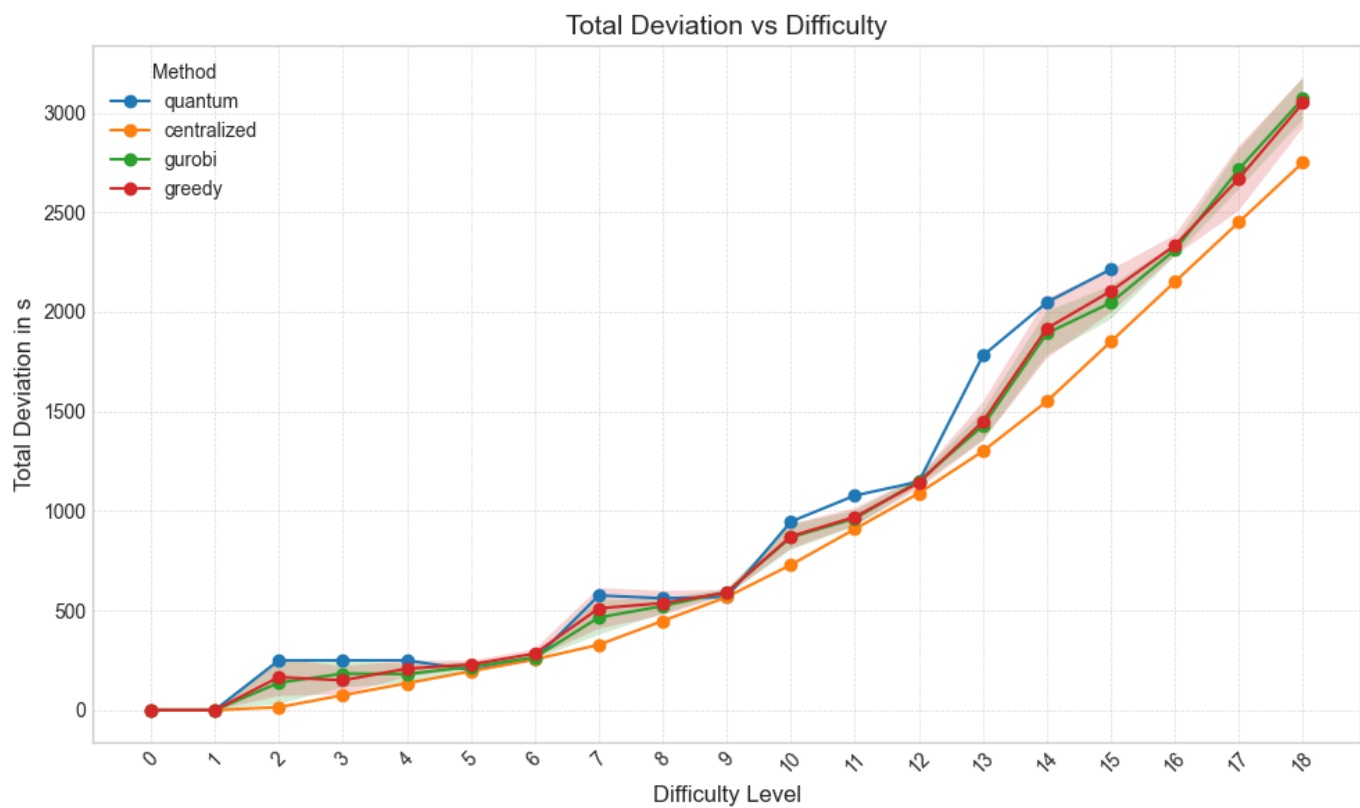


Fig. 6. Average total deviation versus difficulty level.

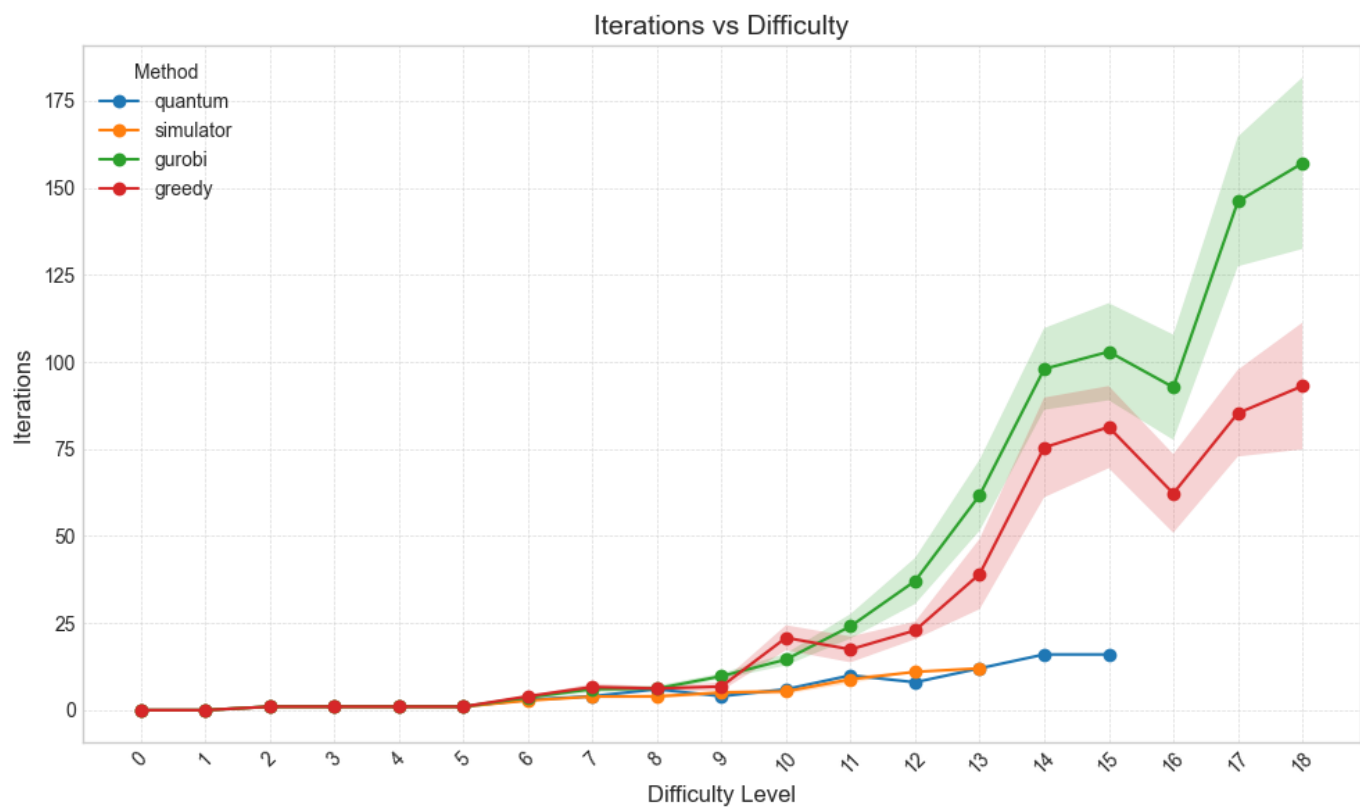


Fig. 7. Average iteration count versus difficulty level.

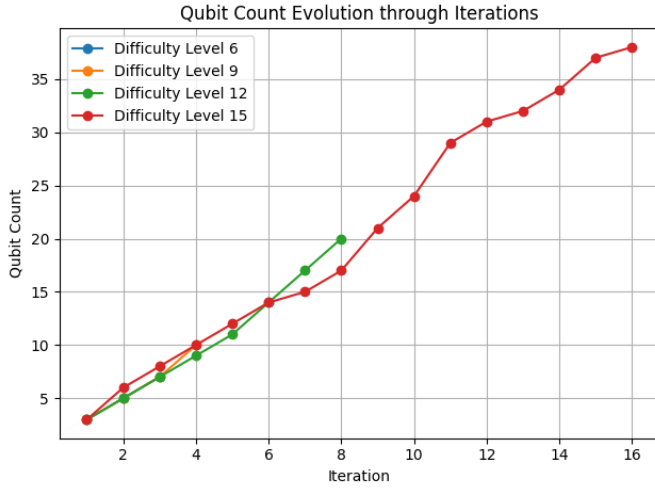


Fig. 8. Qubit count evolution through Benders iterations for difficulty levels 6, 9, 12 and 15.

master-loop iterations than the greedy or Gurobi solver, indicating a more effective selection of constraints to ignore in each iteration.

In practice, QAOA+ does not always return a minimal cover of conflicts—limited circuit depth, shot noise, and a short COBYLA budget bias the distribution toward selecting slightly larger relaxation sets. More constraints are disabled in a single Benders round than strictly necessary. With a larger portion of the search space pruned, the SMT solver call encounters fewer new incompatibilities, so the decomposition terminates in fewer iterations.

In contrast, Gurobi solver delivers a provably minimal cover, and the greedy solver often approaches minimality; many constraints therefore remain active, additional conflicts are revealed, and cut generation must repeat—explaining their sharply rising iteration counts at higher difficulties.

In general minimality of the master cover is not strongly correlated with schedule quality as observed in [3], where even near-random or heuristic covers can yield similar total deviations to the optimal set cover.

D. Quantum Resource and scalability

The growth of qubit requirements over master-loop iterations is plotted in Figure 8 for selected difficulty levels. As more conflicts are added, the circuit must encode larger SCP instances, causing a near-linear increase in total qubits (problem + ancilla).

E. Summary

Figure 9 shows the peak qubit usage across difficulties, rising up to about 38 qubits at level 15 — well below the 133-qubit capacity of IBMQ’s *ibm_torino*. Allocating additional hardware runtime could extend experiments to even higher difficulties.

Taken together, these results highlight a clear trade-off: the centralized solver yields the best schedule quality at the cost

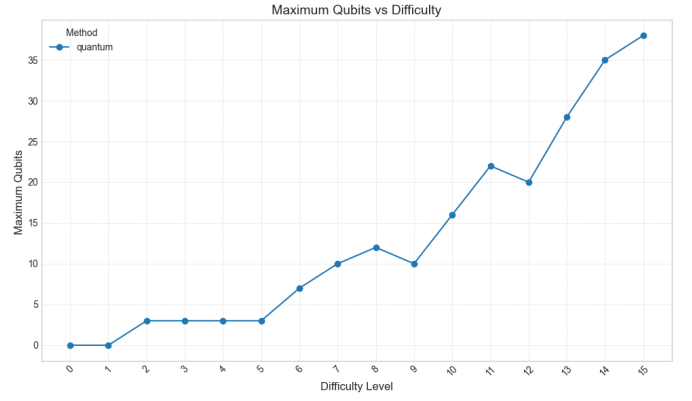


Fig. 9. Maximum qubit usage versus difficulty level for QAOA+.

of runtime, while the greedy approach is fastest but least accurate. Gurobi balances quality and speed. Our QAOA+ method stands out by achieving near-optimal deviation in significantly fewer iterations, with qubit demands that remain within current hardware limits—albeit with significant higher runtimes due to well known NIQS-related limitations.

F. Limitations

Below we highlight some of the main limitations of the present work, which should be interpreted more as a proof of concept than a production decision-support tool.

First, the quantum simulations are performed on the noise-free AerSimulator simulator, which takes exponentially more time than the real machine. On the other end the experiments on physical hardware suffer from decoherence, read-out bias, and CNOT infidelity, and for denser networks the required circuit depth may exceed the device’s coherence window.

Moreover, our current QAOA+ implementation maps one problem qubit per SCP subset plus one ancilla qubit per conflict, restricting us to instances whose total number does not exceed the amount of qubit available (133 on *ibm_torino*). Future work could investigate more efficient mixer Hamiltonians that reduce the need for ancilla qubits.

On the classical side, we cap parameter optimisation at ten COBYLA iterations with ten shots per evaluation. For larger problems, gradient-based or Bayesian methods or more frugal estimators may converge faster or require fewer circuit evaluations. From a modelling perspective, the translator assumes deterministic running times and uses a uniform deviation bucket size $\delta_{\max}/K$; adapting the discretisation grid to actual headway distributions or train priorities could reduce the number of Benders iterations. Finally, although the SMT subproblem scales linearly with event count, it relies on a single-threaded Z3 invocation. Employing a parallel SMT portfolio or a hybrid CP–SMT back-end could yield additional speed-ups on larger industrial instances.

VII. CONCLUSION

This paper presents a quantum-hybrid Benders decomposition approach for tackling the challenging train rescheduling

problem. The method effectively decouples continuous timing decisions from combinatorial precedence constraints, translating industrial data into a tractable optimization workflow. A Z3-based SMT subproblem generates Benders cuts from detected conflicts, which in turn feed into a Set Covering Problem (SCP) master problem solved using the Quantum Alternating Operator Ansatz (QAOA+). This decomposition strategy enables the SCP’s combinatorial complexity to be handled by quantum algorithms, while QAOA+’s feasibility-preserving mixers keep the quantum evolution within the solution subspace, yielding more efficient circuits.

Our experimental results demonstrate several promising aspects of this hybrid approach. While the current implementation does not yet offer a runtime advantage, probably due to quantum processor execution times, the quantum solver requires fewer Benders iterations than its classical counterparts. This suggests that as quantum hardware matures, the iteration efficiency of quantum methods could translate into significant runtime gains. Solution quality across methods remains comparable: the quantum-hybrid approach achieves near-optimal deviations similar to classical solvers, even for denser networks. Moreover, qubit demands grow linearly with problem complexity and remain comfortably within the limits of today’s NISQ devices, such as IBM Torino. This indicates that more complex train rescheduling problems are already solvable on existing hardware, reinforcing the practical potential of this approach.

However, this work remains a proof of concept and several limitations must be addressed before real-world deployment. Current quantum simulations are performed on noise-free emulators, which are computationally expensive, while physical quantum devices still suffer from decoherence, read-out bias, and gate infidelities. For denser networks, circuit depth may exceed coherence windows. Additionally, our QAOA implementation requires one problem qubit per SCP subset and one ancilla qubit per conflict, which constrains scalability. Future research should explore more efficient mixer Hamiltonians to reduce ancilla requirements and enable larger instances. On the classical side, more advanced parameter optimization methods, such as gradient-based or Bayesian techniques, could accelerate convergence. Refinements in problem modeling, such as adapting the discretization grid to headway distributions and employing parallel SMT solvers, could also improve overall performance.

This pioneering integration of QAOA+ within a Benders decomposition framework underscores the potential of quantum computing, even in the NISQ era, to contribute meaningfully to industrial-scale scheduling challenges.

AUTHOR CONTRIBUTIONS

Daniel Kratzer wrote sections II, IV-B4 and IV-B7, and the introduction paragraph at the beginning of Section IV.

Andrei Zubarev wrote the introduction to the Section V and the Section IV-B (except the parts IV-B4, IV-B6 and IV-B7).

Emanuele Poggi wrote Abstract, Section IV-A, Section V from V-A on, and Section VI.

Kilian Krüger wrote Section I, parts of the subsections III-C and III-E, and IV-B6

Moritz Heidtmann wrote parts of III, specifically III-A, III-B, III-D, III-F and Section VII.

REFERENCES

- [1] Deutsche Bahn AG. Integrated report 2024. https://ibir.deutschebahn.com/2024/fileadmin/downloads/DB_IB24_e_web_01.pdf, 2024. Accessed on 2024-06-25.
- [2] Valentina Cacchiani, Dennis Huisman, Martin Kidd, Leo Kroon, Paolo Toth, Lucas Veelenturf, and Joris Wagenaar. An overview of recovery models and algorithms for real-time railway rescheduling. *Transportation Research Part B: Methodological*, 63:15–37, 2014.
- [3] Florin Leutwiler and Francesco Corman. Set covering heuristics in a benders decomposition for railway timetabling. *Computers & Operations Research*, 159:106339, 2023.
- [4] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv e-prints*, pages arXiv-1411, 2014.
- [5] Florin Leutwiler and Francesco Corman. A logic-based benders decomposition for microscopic railway timetable planning. *European Journal of Operational Research*, 303(2):525–540, 2022.
- [6] Stuart Hadfield, Zhihui Wang, Bryan O’gorman, Eleanor G Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.
- [7] JE Beasley. Algorithms for unconstrained two-dimensional guillotine cutting. *Journal of the Operational Research Society*, 36(4):297–306, 1985.
- [8] Egon Balas and Andrew Ho. Set covering algorithms using cutting planes, heuristics, and subgradient optimization: a computational study. *Combinatorial optimization*, pages 37–60, 1980.
- [9] Vasek Chvatal. A greedy heuristic for the set-covering problem. *Mathematics of operations research*, 4(3):233–235, 1979.
- [10] J Benders. Partitioning procedures for solving mixed-variables programming problems. *Computational Management Science*, 2(1), 2005.
- [11] Camille Grange, Michael Poss, and Eric Bourreau. An introduction to variational quantum algorithms for combinatorial optimization problems. *4or*, 21(3):363–403, 2023.
- [12] Camille Grange. *Design and application of quantum algorithms for railway optimisation problems*. PhD thesis, Université de Montpellier, 2024.
- [13] Krzysztof Domino, Emery Doucet, Reece Robertson, Bartłomiej Gardas, and Sebastian Deffner. On the baltimore light rail link into the quantum future. *arXiv e-prints*, pages arXiv-2406, 2024.
- [14] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.
- [15] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.
- [16] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [17] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. Gurobi Optimization, LLC, Houston, TX, 9.1 edition, 2020. Accessed: 2025-07-20.
- [18] Guangtun Zhu. Setcoverpy: A heuristic solver for the set cover problem. *Astrophysics Source Code Library*, pages ascl-2203, 2022.